Data types in Python

16 May 2026 13:29

Number: In number data types stores numerical values only. It is further classified into three different types: int, float and complex.

Type/Class	Description	Examples
int	Integer number	-12, -3, 0, 125, 2
float	Real or floating pointe numbers	-204.5, 4.0, 14.23
complex	Complex numbers	4+5j, 0j, 2-2j

Boolean data type (bool) is a subtype of Integer. It is a unique data type consisting of two constants, True and False. Boolean True value is non-zero, non-null and non-empty. Boolean False is the value zero.
For example:

```
>>>num1 = 10
>>>type(num1)
   <class 'int'>
>>>num2 = -1270
>>>type(num2)
   <class 'int'>
>>>var1 = True
>>>type(var1)
   <class 'bool'>
>>>fl1 = -9.419
>>>type(fl1)
   <class 'float'>
>>>c = 4 + 5j
>>>type(c)
   <class 'complex'>
>>>d = 0j
>>>type(d)
   <class 'complex'>
>>>
```

Sequence

A Python sequence is an ordered collection of items, where each item is indexed by an integer. The three types of sequence data type available in Python are strings, lists and tuples.

i. **String:** String is a group of characters. These characters may be alphabet, digits or special characters including spaces. String values are enclosed either in single quotation mark. Or in double quotation mark. For example: 'Hello' or "Hello". The quotes are not a part of the string, they are used to mark the beginning and end of the string for the interpreter. For example:

```
>>>str1 = 'Hello Friends'
>>>str2 = "990"
>>>type(str1)
   <class 'str'>
>>>type(str2)
   <class 'str'>
>>>
```

ii. **List:** List is a sequence of items separated by commas and the items are enclosed in brackets[].

For example:

```
>>> fruitList = ['Apple', 'Mangoes', "Grapes", "Litches"]
>>> fruitList
['Apple', 'Mangoes', 'Grapes', 'Litches']
>>> type(fruitList)
<class 'list'>
```

iii. **Tuple:** Tuple is a sequence of items separated by commas and items are enclosed in parenthesis (). This is unlike list where the values are enclosed in brackets []. Once created, we cannot change the tuple.
For example:

```
>>> t = (1, 2, 3.5, 4+5j, 'Hello', 3.6, 'a')
>>> t
(1, 2, 3.5, (4+5j), 'Hello', 3.6, 'a')
>>> type(t)
<class 'tuple'>
```

Set

Set is an unordered collection of items separated by commas, and the items are enclosed and curly brackets {}. A set is a similar to list except that it cannot have duplicate entries. Once created, elements of the set cannot be changed. Ordering of elements is not preserved.
For example:

```
>>> set1 = {10, 20.5, "New Delhi"}
>>> type(set1)
<class 'set'>
>>> set1
{'New Delhi', 10, 20.5}
>>>
```

None: None is a special data type with single value. It is used to signify the absence of value in a situation. None supports no special operation and it is neither false nor 0(zero).
For example:

```
myvar = None
type(myVar)

<class 'NoneType'>
```

Mapping: Mapping is an unordered data type in Python. Currently there is only one standard mapping data type in Python called dictionary.

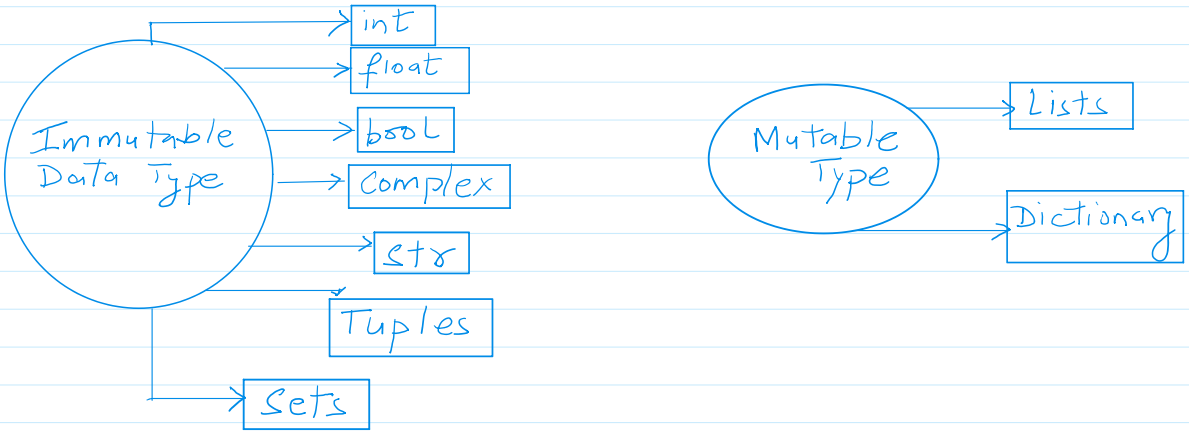
Dictionary: Dictionary in Python holds data items in **key:value** pairs. Items in dictionaries are enclosed in curly brackets{}. Dictionaries permit faster access to data. Every key is separated from its value using (:) sign. The key:value pair of a dictionary can be accessed using the key. The keys are usually strings and their values can be any data type. In order to access any value in the dictionary we have to specify its key in square brackets, that is [].

```
>>>dict1 = {'Fruit': 'Apple', 'Climate':'cold', 'Price(kg)': 120}
>>>dict1
{'Fruit': 'Apple', 'Climate': 'cold', 'Price(kg)': 120}
>>>dict1['Climate']
'cold'
>>>dict1['Price(kg)']
120
```

Mutable and Immutable Data types

Sometimes we may require to change or update the values of certain variables used in the program. However, for a certain database, Python does not allow us to change the values once a variable of that data type has been created and assign values.

Variables whose values can be changed after they are created and assigned are called **mutable**. So variables whose values cannot be changed after they are created and assigned are called **immutable**. When an attempt is made to update the value of an immutable variable, the old variable is destroyed and a new variable is created by the same name in memory.



Classification of data types

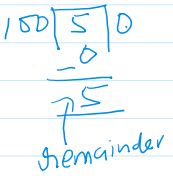
Operators

An operator is used to perform specific mathematical or logical operation on values. The values of the operator work on are called **operands**. For example, in the expression 10 + NUM, The value 10 and the variable NUM are operands and the + (plus) sign is an Operator. Python supports several kind of operators whose categorization is briefly explained below:

Arithmetic Operators

Operator	Operation	Description	Example
+	**Addition**	Adds two numeric values on either side of the operator. This operator can also be used to concatenate two strings on either side of the operator.	**n1 = 5** **n2 = 3** **r = n1 + n2** **r** **8** **s1 = 'Aarav'** **s2 = 'Tebrewal'** **name = s1 + ' ' + s2** **name** **'Aarav Tebrewal'**

-	**Subtraction**	Subtract the operand on the right from the operand on the left.	**n1 5 n2 3 r = n1 - n2 r 2**
*	**Multiplication**	Multiplies the two values on both side of the operator. Repeat the item on the left side of the operator if first operand is a string and 2nd operand is an integer value.	**n1 5 n2 3 p = n1 * n2 p 15 '*' * n1 '*****' n1 * '*' '*****'**
/	**Division**	Divides the operand on the left. By the operand on the right and returns the quotient.	**n1 5 n2 3 q = n1 / n2 q 1.6666666666666667**
%	**Modulus**	Divides the operand on the left of by the operand on the right and returns the remainder.	**n1 5 n2 3 r = n1 % n2 r 2 r = n1 % 100 r 5**
//	**Floor Division**	Divides the operand on the left by the operand on the right, and returns a quotient by removing the decimal part. It is sometimes also called integer division.	**r 5 n1 5 n2 3 r = n1 // n2 r 1**
**	**Exponent**	Performs exponential (power) calculation on operands. That is, raise the operand on the left to the power of the operand on the right.	**n1 5 n2 3 r = n1 ** n2 r 125**



Relational Operators

A relational operators compare the values of the operands on its either side and determines the relationship among them.

Assume the Python variables:

```
>>> num1 = 10
>>> num2 = 0
>>> num3 = 10
>>> str1 = "Good"
>>> str2 = "Afternoon"
```

Relational operators in Python

Operator	Operation	Description	Example
==	Equals to	If the values of two operands are equal, then the condition is true, otherwise it is false.	num1 == num2 False num1 == num3 True str1 == str2 False
!=	Not equal to	If values of two operands are not equal then condition is true, otherwise it is false.	num1 != num2 True num1 != num3 False str1!=str2 True
>	Greater than	If the value of the left side operand is greater than the value of the right side operand, then condition is true, otherwise it is false.	num1 > num2 True num1 > num3 False
<	Less than	If the value of the left side operand is less than the value of the right side operand, then the condition is True. Otherwise it is False.	num1 < num2 False num1 < num3 False num2 < num3 True
>=	Greater than or equal to	If the value of the left side operand is greater than or equal to the value of the right side operand, then the condition is true. Otherwise it is false.	num1 >= num3 True num1 >= num2 True num2 >= num3 False
<=	Less than or equal to	If the value of the left operand is less than or equal to the value of the right operand, the it is true, otherwise it	num1 <= num3 True num1 <= num2 False num2 <= num3

		is false.	True

Assignment Operators

Assignment operator assigns or changes the value of the variable on its left.

Assignment operator in Python

Operator	Description	Example
=	Assigns a value from right side operand to the left side operand.	num = 100 num10 = num num10 100 num 100 country = 'India' country 'India'
+=	OK it adds the value of right side operand to the left side operand and assigns a result to the left side operand. Note: x += y is same as x = x + y	num1 10 num2 0 num3 10 num 100 num2 += num3 num2 10 num2 += num num2 110
-=	It subtracts the value of right side operand from the left side operand and assigns a result to the left side operand. Note: x -= y is same as x = x - y	num2 110 num2 -= num1 num 100
*=	It multiplies the value of the right side operand with the value of left side operand and assigns the result to the left side operand. Note: x *= y is same as x = x * y	num1 10 num2 100 num3 10 num3 *= num1 num3 100
%=		
//=		
**=		

Logical Operators

There are three logical operators supported by Python. These operators **(and, or, not)** are written in lower case only. These logical operators evaluates to either true or false based on the logical operands on either side. Every value is

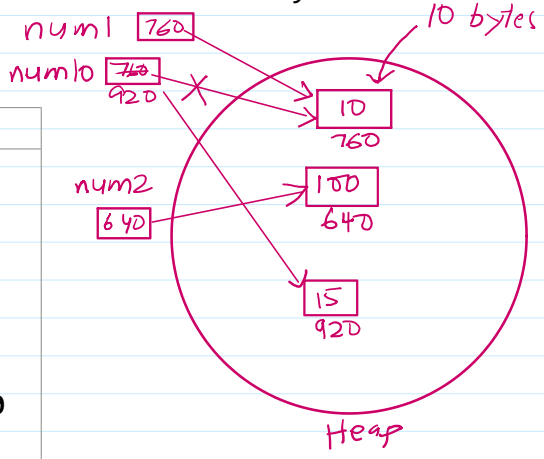
logically either true or false. By default all values are True except None, False 0 (0) empty collections "", (), [], {} and few other special values.

Logical operators in Python

Operator	Operation	Description	Example
and	Logical and	If both the operands are true then condition becomes true.	num1 == num3 and num1 <= num2 False num1 10 num3 100 num2 100 num1 <= num3 and num2 == num3 True
or	Logical or	If any of the two operands are true, then condition becomes true.	num1 == num3 or num2 == num3 True
not	Logical not	Used to reverse the logical state of its operand.	not(num1 == num3 and num1 <= num2) True not(num1 == num3 or num2 == num3) False

Identity Operators

Identity operators are used to determine whether the value of a variable is of a certain type or not. Identity operators can also be used to determine whether two variables are referring to the same object or not. There are two identity operators.



Identity operators in Python

Operators	Description	Example
is	Evaluates true if the variable on either side of the operator points towards the same memory location. And false otherwise.	num1 10 num10 10 num1 is num10 True id(num1) 140725955556760 id(num10) 140725955556760
is not	Evaluates to false if the variables on either side of the operator points to the same memory location. And true otherwise.	num1 is not num10 False num1 is not num2 True id(num1) 140725955556760 id(num2) 140725955559640 num2 100

Membership Operators

Membership operators are used to check if a value is a member of the given sequence or not.

Membership operators in Python

Operator	Description	Example
in	**Returns true if the variable or value is found in the specified sequence, and false otherwise.**	**li1** **[10, 12, 8, 100, 29, 64]** **18 in li1** **False** **19 in li1** **False** **'4' in li1** **False** **12 in li1** **True**
not in	**Returns true if the variable or value is not found in the specified sequence, and false otherwise.**	**li1** **[10, 12, 8, 100, 29, 64]** **18 not in li1** **True** **100 in li1** **True** **100 not in li1** **False**

Precedence of Operators

Evaluation of the expression is based on precedence of operators. When an expression contains different kinds of operators, precedence determines which operator should be applied first. Higher precedence of operator is evaluated before the lower precedence operator. Most of the operators studied till now are binary operators. Binary operators are operators with two operands. The unary operator needs only one operand, and they have the higher precedence than binary operators. The minus as well as plus operators can act as both unary and binary operators. But **not** is a unary logical operator.

Here's the **complete table of Python operator precedence** from **highest to lowest**, along with associativity, based on the official precedence rules 1 2:

Precedence (High → Low)	**Operators**	**Description**	**Associativity**
1	()	Parentheses (grouping)	Left-to-right
2	x[index], x[index:index], x(...), x.attr	Subscription, slicing, function call, attribute reference	Left-to-right
3	await x	Await expression	Left-to-right
4	**	Exponentiation	**Right-to-left**
5	+x, -x, ~x	Unary plus, unary minus, bitwise NOT	Right-to-left
6	*, @, /, //, %	Multiplication, matrix	Left-to-

		multiplication, division, floor division, modulus	right
7	+, -	Addition, subtraction	Left-to-right
8	<<, >>	Bitwise shift operators	Left-to-right
9	&	Bitwise AND	Left-to-right
10	^	Bitwise XOR	Left-to-right
11	`	`	Bitwise OR
12	in, not in, is, is not, <, <=, >, >=, !=, ==	Comparisons, membership, identity tests	Left-to-right
13	not x	Logical NOT	Right-to-left
14	and	Logical AND	Left-to-right
15	or	Logical OR	Left-to-right
16	if ... else ...	Conditional expression	Right-to-left
17	lambda	Lambda expression	Left-to-right
18	:=	Assignment expression (Walrus operator)	Right-to-left

Key Notes 1 2:

- **Parentheses** can always be used to override default precedence.
- **Exponentiation (**)** is **right-associative**: 2 ** 3 ** 2 → 2 ** (3 ** 2) = 512.
- **Most operators** are left-associative.
- **Comparisons** can be chained: x < y < z is equivalent to (x < y) and (y < z).
- **Assignment operators** (=, +=, etc.) are **not** in this table because they are statements, not expressions, and have their own rules.

Got it ☑
Python does not have a built-in function to directly list **all operators with precedence**, but we can create a **reference table** in code and print it nicely.
Here's a **self-contained Python script** that prints the precedence table exactly as per Python's official rules.

Python

```
#!/usr/bin/env python3
"""
Python Operator Precedence Table Printer
Author: ChatGPT
Description: Prints Python's operator precedence from highest to lowest.
"""
from textwrap import dedent
def get_operator_precedence_table():
    """
    Returns a list of tuples: (precedence_level, operators, description,
associativity)
```

```
    Based on Python's official precedence rules.
    """
    return [
        (1, "()", "Parentheses (grouping)", "Left-to-right"),
        (2, "x[index], x[index:index], x(...), x.attr", "Subscription, slicing,
function call, attribute reference", "Left-to-right"),
        (3, "await x", "Await expression", "Left-to-right"),
        (4, "**", "Exponentiation", "Right-to-left"),
        (5, "+x, -x, ~x", "Unary plus, unary minus, bitwise NOT", "Right-to-left"),
        (6, "*, @, /, //, %", "Multiplication, matrix multiplication, division,
floor division, modulus", "Left-to-right"),
        (7, "+, -", "Addition, subtraction", "Left-to-right"),
        (8, "<<, >>", "Bitwise shift operators", "Left-to-right"),
        (9, "&", "Bitwise AND", "Left-to-right"),
        (10, "^", "Bitwise XOR", "Left-to-right"),
        (11, "|", "Bitwise OR", "Left-to-right"),
        (12, "in, not in, is, is not, <, <=, >, >=, !=, ==", "Comparisons,
membership, identity tests", "Left-to-right"),
        (13, "not x", "Logical NOT", "Right-to-left"),
        (14, "and", "Logical AND", "Left-to-right"),
        (15, "or", "Logical OR", "Left-to-right"),
        (16, "if ... else ...", "Conditional expression", "Right-to-left"),
        (17, "lambda", "Lambda expression", "Left-to-right"),
        (18, ":=", "Assignment expression (Walrus operator)", "Right-to-left"),
    ]
def print_precedence_table():
    table = get_operator_precedence_table()
    print(f"{'Precedence':<10} {'Operators':<50} {'Description':<60}
{'Associativity'}")
    print("-" * 140)
    for prec, ops, desc, assoc in table:
        print(f"{prec:<10} {ops:<50} {desc:<60} {assoc}")
if __name__ == "__main__":
    print("\nPython Operator Precedence Table (Highest → Lowest)\n")
    print_precedence_table()
    print("\nNote: Parentheses can always be used to override precedence.")
```

How to Run:
1. Save this as precedence_table.py.
2. Run in terminal:

```
Bash
python3 precedence_table.py
```

3. You'll get a **formatted precedence table** printed in your console.

Input and output

Sometimes a program needs to interact with the users to get some input data or information from the end user and process it to give the desired output. In Python we have the **input()** function for taking the user input. The **input()** function prompts the user to enter data. It accepts all user input as a string. The user may enter a number or a string but the **input()** function treats them as string only. The syntax for **input()** is:

```
input('[Prompt]')
```

Prompt is the string we may like to display on the screen prior to taking the input, and it is optional. When a prompt is specified first, it is displayed on the

screen, after which the user can enter data. The input () takes exactly what is typed from the keyboard, converts it into string and assign it to the variable on the left hand side of the assignment operator (=). Entering data for the input function is terminated by pressing the Enter key.

Example:

```
name = input("Enter your name: ")
Enter your name: Shiv
age = int(input("Enter your age: "))
Enter your age: 43
age
43
name
'Shiv'
type(name)
<class 'str'>
type(age)
<class 'int'>
```

Python uses the print () function to output data to standard output device- the screen. The function print () evaluates the expression before displaying it on the screen. The print () outputs the complete line and then moves to the next line for subsequent output. The syntax for print () is:

```
print(value[,…, sep = ' ', end = '\n'])
```

- **sep:** The optional parameter **sep** is a separator between the output values. We can use a character, integer or a string as a separator. The default separator is a space.
- **end:** This is also optional and it allows us to specify any string to be appended after the last value. The default is a new line.

For example:

```
print(name)
Shiv
print('Hello!',name)
Hello! Shiv
print('Hello!',name, age)
Hello! Shiv 43
print("India"+"is"+"my"+"country")
Indiaismycountry
print("India","is","my","country")
India is my country
```

Type conversion

Explicit Type Conversion

Explicit conversions are also called. Type casting happens when the data type conversion takes place because the programmer forced it in the program. The general form of an explicit data type conversion is:

```
New_data_type(expression)
```

With explicit type conversion there is a risk of loss of information since we are focus forcing an exception to be of a specific type.

For example:

```
x = 20.89
```

```
type(x)
<class 'float'>
y = int(x)
y
20
```

Explicit type conversion functions in Python

Function	Description
int(x)	**Convert X to an integer.**
float(x)	**Converts X to a floating point number.**
str(x)	**Convert X to a string representation.**
chr(x)	**Convert X to a character.**
unichar(x)	**Converts X to a Unicode character.**

Implicit Conversion
Implicit conversions, also known as coercion, happens when data type conversion is done automatically by Python and it is not instructed by the programmer.

Debugging: A programmer can make mistakes while writing a program and hence the program may not execute or may generate wrong output. The process of identifying and removing such mistakes, also known as bugs or errors, from a program, is called debugging. Errors occurring in the program can be categorized as:
1. Syntax errors
2. Logical errors
3. Runtime errors